

# TASK 1 – DESIGN THE ALGORITHM

## 1. Problem Definition

The university maintains a large database containing student records, each identified by a unique Registration Number. The objective is to design efficient algorithms to locate a given registration number in the database.

The database size may increase from 1,000 records to 10,000, 100,000 and 1,000,000 or more records. Therefore, different searching strategies must be investigated to identify the most suitable approach for small as well as large datasets.

Three different algorithmic strategies are proposed: Linear Search, Recursive Binary Search and Hash Table Search. These approaches are selected because they use different methods to locate records and have different preprocessing requirements and performance characteristics.

## 2. Algorithm 1 – Linear Search

### Basic Idea

Linear Search is a sequential searching technique that examines each registration number one by one, starting from the first record. Each registration number is compared with the required key. If a match is found, the algorithm returns the corresponding record. If the end of the database is reached without finding a match, the algorithm reports that the record does not exist.

### Design Strategy

Linear Search follows the **Brute Force strategy** because it checks the records sequentially without using any special structure or eliminating groups of records in advance.

### Pseudocode

1. Start.
2. Begin with the first record.

3. Compare the current registration number with the required registration number.
4. If both are equal, return “Record Found”.
5. Otherwise, move to the next record.
6. Repeat until the record is found or all records have been checked.
7. If no match is found, return “Record Not Found”.
8. Stop.

### **Preprocessing Required**

No preprocessing is required. The registration numbers can be stored in any order, and sorting is not necessary.

### **Justification for Selection**

Linear Search is selected because it is simple to understand and implement. It works with both sorted and unsorted data and provides a baseline for comparing the performance of other algorithms. However, it may require examining every record in a large database, making it less suitable for millions of records.

## **3. Algorithm 2 – Recursive Binary Search**

### **Basic Idea**

Binary Search locates a registration number by repeatedly dividing the search range into two halves. It compares the required registration number with the middle element of the current range.

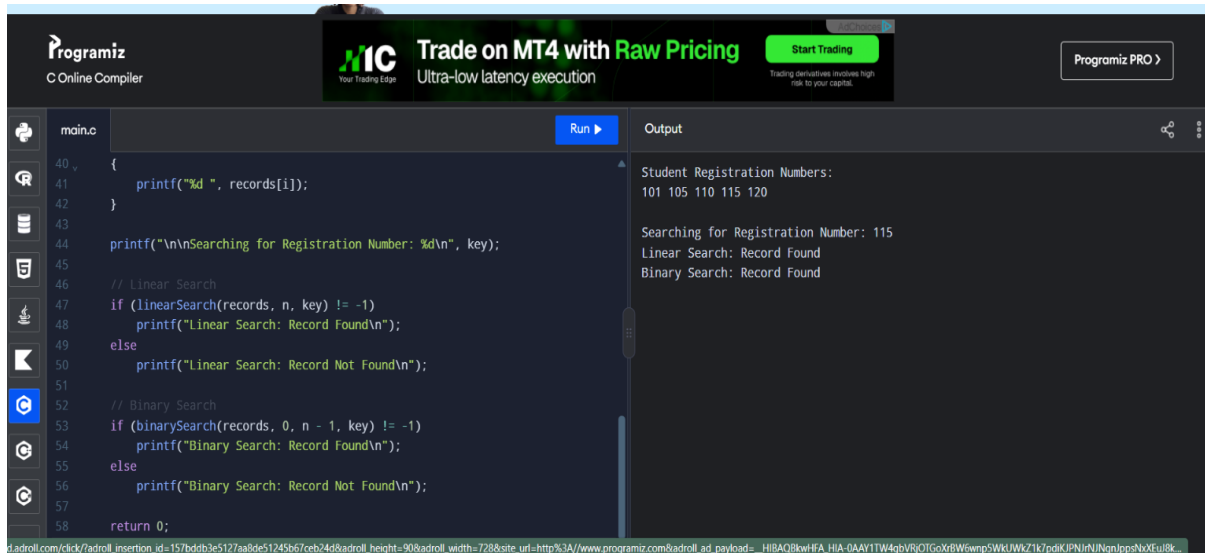
If the middle element matches the key, the record is found. If the key is smaller, the algorithm searches the left half. If the key is larger, it searches the right half. This process continues recursively until the record is found or the search range becomes empty.

### **Design Strategy**

Binary Search follows the **Divide and Conquer strategy**. It divides the original problem into a smaller subproblem and continues searching only within the relevant half of the data.

## Pseudocode

1. Start.
2. Check whether the search range is empty.
3. If the range is empty, return “Record Not Found”.
4. Find the middle element of the current search range.
5. If the middle registration number equals the required key, return “Record Found”.
6. If the key is smaller than the middle element, recursively search the left half.



The screenshot displays the Programiz C Online Compiler interface. The editor shows a C program named `main.c` that implements a binary search algorithm. The program prints the student registration numbers (101, 105, 110, 115, 120) and then searches for the registration number 115. The output window shows the results: "Student Registration Numbers: 101 105 110 115 120", "Searching for Registration Number: 115", "Linear Search: Record Found", and "Binary Search: Record Found".

```
main.c
40 {
41     printf("%d ", records[i]);
42 }
43
44 printf("\n\nSearching for Registration Number: %d\n", key);
45
46 // Linear Search
47 if (linearSearch(records, n, key) != -1)
48     printf("Linear Search: Record Found\n");
49 else
50     printf("Linear Search: Record Not Found\n");
51
52 // Binary Search
53 if (binarySearch(records, 0, n - 1, key) != -1)
54     printf("Binary Search: Record Found\n");
55 else
56     printf("Binary Search: Record Not Found\n");
57
58 return 0;
```

Output

```
Student Registration Numbers:
101 105 110 115 120

Searching for Registration Number: 115
Linear Search: Record Found
Binary Search: Record Found
```